

编译器设计专题实验报告

实验六：中间代码生成

张凌曜 2234214014 计算机 2306

1 实验目的

写一个中间代码生成器,把源程序翻译成四元式。四元式格式是 $(op, arg1, arg2, result)$, 表示 $result = arg1 \ op \ arg2$, 是编译器中间阶段的常用表示。

2 实验原理

表达式 $c = a + b * 2$ 翻译成四元式:

op	arg1	arg2	result
*	b	2	t1
+	a	t1	t2
=	t2		c

if 语句用条件跳转 (JZ) 实现, while 还要在循环体末尾加一个无条件跳转 (JMP) 回到循环头。

3 实验过程

3.1 表达式翻译

用递归下降, `parseExpr` 处理加减, `parseTerm` 处理乘除, 自底向上翻译。每次遇到二元运算就分配一个临时变量 t_i , 生成一条四元式。乘法在 `parseTerm` 里先处理, 所以优先级自然是对的。

3.2 控制流翻译

if-else 需要生成两个标签：L0 给 else 入口，L1 给结束位置。条件为假跳 L0，if 体执行完跳 L1，然后 L0 开始 else 体，L1 结束。

while 类似但多了个回跳：L0 标记循环头，条件判断后 JZ 跳出到 L1，循环体末尾 JMP 回 L0。

4 测试结果

测试程序：

```
1 int main() {
2     int a;
3     int b;
4     int c;
5     a = 10;
6     b = 20;
7     c = a + b * 2;
8     if (c > 30) {
9         print c;
10    } else {
11        print a;
12    }
13    while (a > 0) {
14        a = a - 1;
15    }
16    return 0;
17 }
```

生成的四元式：

看几个关键点：006-008 是 $c = a + b * 2$ ，先算乘法再算加法，优先级没问题。010-015 是 if-else，JZ 跳 else，JMP 跳过 else，标签位置都正确。016-022 是 while，LABEL 标循环头，JZ 跳出，JMP 回跳。

#	op	arg1	arg2	result
000	FUNC	main		
001-003	DECL	a,b,c	int	
004	=	10		a
005	=	20		b
006	*	b	2	t1
007	+	a	t1	t2
008	=	t2		c
009	>	c	30	t3
010	JZ	t3		L0
011	PRINT	c		
012	JMP			L1
013	LABEL	L0		
014	PRINT	a		
015	LABEL	L1		
016	LABEL	L2		
017	>	a	0	t4
018	JZ	t4		L3
019	-	a	1	t5
020	=	t5		a
021	JMP			L2
022	LABEL	L3		
023	RETURN	0		
024	ENDFUNC	main		

```
===== 符号表 =====
main : int (函数)
a : int
b : int
c : int

===== 四元式中间代码 =====
编号 ( 运算符, 操作数1, 操作数2, 结果 )
-----
000 ( FUNC      , main      ,      ,      )
001 ( DECL      , a        , int   ,      )
002 ( DECL      , b        , int   ,      )
003 ( DECL      , c        , int   ,      )
004 ( =         , 10       ,      , a    )
005 ( =         , 20       ,      , b    )
006 ( *         , b        , 2     , t1   )
007 ( +         , a        , t1    , t2   )
008 ( =         , t2       ,      , c    )
009 ( >         , c        , 30    , t3   )
010 ( JZ        , t3       ,      , L0   )
011 ( PRINT     , c        ,      ,      )
012 ( JMP       ,          ,      , L1   )
013 ( LABEL    , L0       ,      ,      )
014 ( PRINT     , a        ,      ,      )
015 ( LABEL    , L1       ,      ,      )
016 ( LABEL    , L2       ,      ,      )
017 ( >         , a        , 0     , t4   )
018 ( JZ        , t4       ,      , L3   )
019 ( -         , a        , 1     , t5   )
020 ( =         , t5       ,      , a    )
021 ( JMP       ,          ,      , L2   )
022 ( LABEL    , L3       ,      ,      )
023 ( RETURN   , 0        ,      ,      )
024 ( ENDFUNC  , main     ,      ,      )

共 25 条四元式
Press any key to continue . . . |
```

用代码库的 1.src 也跑了一下，能正确生成四元式。

```
===== 符号表 =====
main : int (函数)
x : int

===== 四元式中间代码 =====
编号      ( 运算符, 操作数1, 操作数2, 结果 )
-----
000      ( FUNC      , main      ,          ,          )
001      ( DECL      , x        , int      ,          )
002      ( =         , 5        ,          , x        )
003      ( RETURN    , 0        ,          ,          )
004      ( ENDFUNC   , main     ,          ,          )
005      ( CALL      , main     ,          ,          )

共 6 条四元式
Press any key to continue . . . |
```

5 实验总结

四元式生成的核心就是给每个运算分配临时变量，控制流用标签加跳转处理。这六个实验串起来其实就是一条编译链：DFA 定义词法规则 → 词法分析出 Token → 语法分析建 AST → 语义分析加符号表 → 中间代码生成四元式，每一步的输出就是下一步的输入。